Technical Documentation — Tri-Model Spatio-Temporal Ensemble (Traffic Demand Problem)

1. Executive Summary

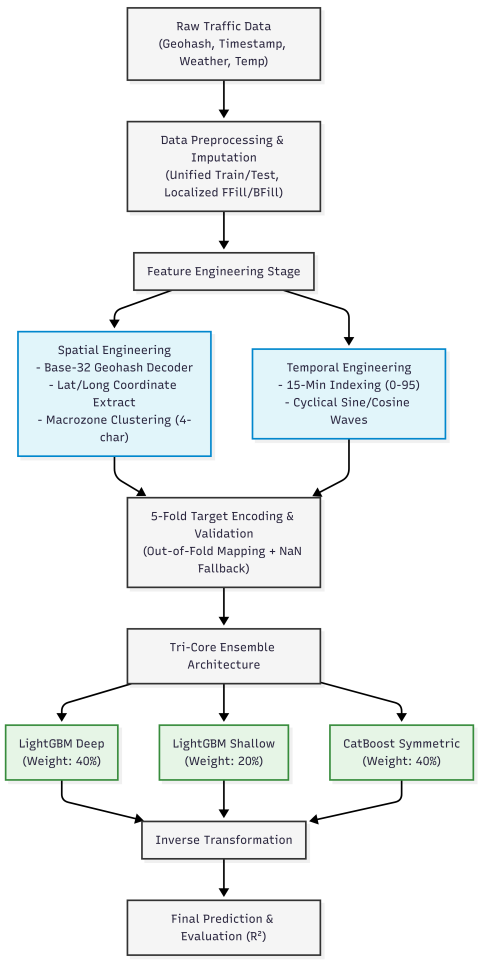
This document outlines an end-to-end machine learning pipeline to predict traffic demand with high precision. The system is built around a **Tri-Model Spatio-Temporal Ensemble** that combines:

- **Granular spatial decoding** (geohash → latitude/longitude + macrozone features)
- **Cyclical temporal tracking** (96-slot daily cycle encoded with sine/cosine)
- **Regularized ensembling** of gradient-boosting models (LightGBM + CatBoost)

The pipeline mitigates leakage using localized cross-validation for target encodings and improves robustness via tiered fallback imputation for missing/unseen values.

---

2. System Architecture Overview



3. Data Preprocessing & Robust Imputation

To guarantee consistency across training and inference and to handle missing data reliably:

- **Unified processing:** Training and test datasets are temporarily concatenated prior to feature engineering. This ensures categorical mappings, encodings, and imputation states remain identical across distributions and prevents structural mismatches at inference time.
- **Granular imputation:** Meteorological fields (e.g., Temperature, Weather) are imputed using `ffill`/`bfill` **grouped by geohash zones**, preserving localized micro-climate patterns instead of washing them out with global summary statistics.

---

4. Feature Engineering

4A. Spatial Engineering (Decoding the Map)

- **Native geohash decoder:** A custom Base-32 binary decoder converts geohash strings into continuous **latitude/longitude** without external spatial libraries.

- **Geometric boundaries:** Continuous coordinates allow tree-based learners to create clean spatial split thresholds aligned with real geographic boundaries.
- **Neighbourhood clustering:** A **macrozone** feature is created from the first 4 geohash characters to preserve coarse spatial context and improve behavior on unseen exact coordinates.

4B. Temporal Engineering (The 96-Slot Cyclical Wave)

- **15-minute granularity:** Timestamp (hour + minute) is flattened to a timeslot index in **[0, 95]**.
- **Cyclical transforms:** To model circular proximity (23:45 ≈ 00:00), the index is projected using sine/cosine:

$$\text{Time}_{\sin} = \sin\left(\frac{2\pi \cdot \text{timeslot}}{96}\right)$$

$$\text{Time}_{\cos} = \cos\left(\frac{2\pi \cdot \text{timeslot}}{96}\right)$$

---

5. Validation Strategy & Target Encodings

- **Smoothened K-Fold target encoding:** High-cardinality spatial categoricals are encoded via historical demand averages. To avoid leakage, encodings are computed using **5-fold CV** and applied out-of-fold (OOF) only.
- **NaN fallback net:** Unseen categories or sparse groups are handled using a cascading fallback strategy that substitutes missing encodings with global training statistics (e.g., mean, standard deviation), preventing downstream failures.

---

6. Tri-Core Ensemble Architecture

Predictions are combined using a weighted blend (**40% / 20% / 40%**) after applying inverse transformations where required.

Model Core	Ensemble Weight	Design Complexity	Primary Objective
LightGBM Deep	40%	High (`num_leaves=63`)	Memorize and map deep micro-zone spatial/temporal interactions.
LightGBM Shallow	20%	Low (`num_leaves=31`)	Macro-regularizer capturing broader city-level demand trends (different seed).
CatBoost Symmetric	40%	Level-wise symmetric	Native categorical handling via ordered target statistics; regularizes structural variance.

---

7. Metrics & Performance Evaluation

The system is evaluated using the **Coefficient of Determination (R^2)**:

$$R^2 = 1 - \frac{\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}{\sum_{i=1}^{n}(y_i - \bar{y})^2}$$

The multi-model design targets stable variance explained across both high-density peak periods and low-demand late-night intervals.

---

8. Technology Stack & Tools Used

- **Language:** Python
- **Data manipulation & math:** `pandas`, `numpy`, `math`
- **Modeling:** LightGBM (deep + shallow), CatBoost (symmetric / ordered stats)
- **Validation & metrics:** `scikit-learn` (K-Fold, evaluation utilities)
- **Environment:** Linux / Jupyter
